Recovery Debt — Full Build Guide

The technical companion to `PLAN.md`. Every tool, every file, every command, every code skeleton you need to actually build this in 3 weeks.

Table of Contents

1. Tech Stack — Full Picture
2. Architecture Diagram
3. Project Structure
4. Environment Variables
5. Day-by-Day Build (with code)
6. The 3 Differentiation Features (in detail)
7. Deployment
8. Testing Strategy
9. Common Errors & Fixes
10. References

---

1. Tech Stack — Full Picture

Layer	Tool	Version	Why this and not alternatives	Install / setup
Frontend framework	Next.js	14+	Industry standard, native to Vercel, App Router gives you server components for free	`npx create-next-app@latest frontend --ts --tailwind --app`
Language (frontend)	TypeScript	5+	Catches API-shape bugs at compile time	included with Next.js
Styling	Tailwind CSS	3+	No CSS file management, Tailwind classes inline	included
UI components	shadcn/ui	latest	Copy-paste components, full ownership of code	`npx shadcn-ui@latest init`
Charts	Recharts	2+	Composable, clean API, plays well with React	`npm i recharts`
PWA	next-pwa	5+	Adds manifest + service worker to Next.js	`npm i next-pwa`
Frontend hosting	Vercel	-	Free, one-command deploy, auto-preview	`npm i -g vercel`
Backend framework	FastAPI	0.110+	Async, fast, plays with sklearn/pandas, OpenAPI for free	`pip install fastapi uvicorn`
Language (backend)	Python	3.11+	All ML lives here	`python --version`
Database client	psycopg / asyncpg	latest	Direct Postgres access	`pip install psycopg[binary]`
ORM (optional)	SQLAlchemy 2	2+	If you want models, not raw SQL	`pip install sqlalchemy`
DB + Auth	Supabase	-	Postgres + auth + storage in one product	https://supabase.com new project
Backend hosting	Railway	-	Simple Python deploys, supports cron	https://railway.app
ML library	scikit-learn	1.4+	Ridge, pipelines, time-series CV	`pip install scikit-learn`
Data manipulation	pandas	2+	Feature engineering	`pip install pandas`
Numeric	numpy	1.26+	Required by sklearn	`pip install numpy`
Explainability	shap	0.45+	Per-prediction attribution	`pip install shap`
Optimization	scipy	1.11+	`scipy.optimize.minimize` for the inverse planner	`pip install scipy`
WHOOP API client	httpx	0.27+	Async HTTP client	`pip install httpx`
OAuth helpers	authlib	1.3+	Token refresh boilerplate	`pip install authlib`

Layer	Tool	Version	Why this and not alternatives	Install / setup
Webhook signature	hmac (stdlib)	-	Verify WHOOP webhook signatures	built in
Cron / scheduling	APScheduler or Railway cron	-	Nightly retraining at 4 AM local	Railway → Settings → Cron
LLM (Week 3 stretch)	Anthropic SDK	latest	Strict-JSON parser for journal entries	`pip install anthropic`
Push notifications	web-push	-	Web Push API directly (no third party)	`npm i web-push`
Testing (Python)	pytest	8+	Standard	`pip install pytest`
Testing (TS)	Vitest	1+	Fast, Jest-compatible	`npm i -D vitest`
Linting (Python)	ruff	latest	Fast, replaces black + flake8 + isort	`pip install ruff`
Linting (TS)	ESLint + Prettier	-	Comes with Next.js	included

What you do NOT need

- **MongoDB / Redis** — overkill for one user
- **Kafka / RabbitMQ** — webhooks + cron is enough
- **Docker** — Vercel and Railway handle containerization for you
- **Kubernetes** — please no
- **A separate auth provider** — Supabase Auth handles it
- **A custom ML training cluster** — Ridge trains in milliseconds

---

2. Architecture Diagram

```
┌──────────────────┐
│  WHOOP Cloud     │
│  (their servers) │
└──────────────────┘
      │
      │ OAuth + Webhooks + REST
      ▼
┌──────────────────────────────────────────────────────────────┐
│                FastAPI Backend (Railway)                     │
│                                                              │
│  ┌──────────────┐  ┌──────────────┐  ┌──────────────┐       │
│  │ /api/whoop/* │  │ /api/predict │  │ /api/goals   │       │
│  │ OAuth, sync  │  │ Today's      │  │ Inverse      │       │
│  │              │  │ receipt      │  │ planner      │       │
│  └──────────────┘  └──────────────┘  └──────────────┘       │
│        │                │                │                   │
│        ▼                ▼                ▼                   │
│  ┌──────────────────────────────────────────────┐           │
│  │  ml/features.py — build_feature_matrix (pure)  │         │
│  │  ml/train.py    — Ridge + TimeSeriesSplit      │         │
│  │  ml/explain.py  — shap.LinearExplainer         │         │
│  │  ml/solve.py    — scipy.optimize (inverse)     │         │
│  └──────────────────────────────────────────────┘           │
│                          │                                  │
└──────────────────────────┼──────────────────────────────────┘
                           │
                           ▼
                ┌──────────────────────┐
                │  Supabase / Postgres  │
                │                       │
                │  users, whoop_tokens, │
                │  recoveries, cycles,  │
                │  sleeps, workouts,    │
                │  checkins, predictions, │
                │  shap_values, models, │
                │  goals (new)          │
                └──────────────────────┘
                           ▲
                           │ reads only
                           │
┌──────────────────────────┼──────────────────────────────────┐
│                Next.js Frontend (Vercel) — PWA               │
│                                                              │
│   /            (ledger homepage — bank-statement view)       │
│   /receipt/[d] (one day's SHAP itemized receipt)             │
│   /simulator   (what-if sliders for tomorrow)                │
│   /plan        (Inverse Planner — solve-for-Saturday)        │
│   /profile     (Sensitivity Profile — body personality)      │
│   /wallet      (Cumulative SHAP — year-in-review)            │
│   /checkin     (15-second daily form)                        │
│   /demo        (synthetic-data mode for recruiters)          │
└──────────────────────────────────────────────────────────────┘
```

```
Cron jobs (Railway, runs at 4 AM user-local):
   1. safety_pull.py    — re-fetch last 3 days from WHOOP
   2. nightly_train.py  — retrain Ridge, regenerate SHAP, predict tomorrow
   3. push_forecast.py  — send PWA push notif if forecast < threshold
```

---

3. Project Structure

```
recovery-debt/
├── frontend/                              # Next.js 14 app
│   ├── app/
│   │   ├── (dashboard)/                   # protected routes
│   │   │   ├── page.tsx                   # ledger homepage
│   │   │   ├── receipt/[day]/page.tsx     # per-day SHAP detail
│   │   │   ├── simulator/page.tsx         # what-if sliders
│   │   │   ├── plan/page.tsx              # inverse planner ⋆
│   │   │   ├── profile/page.tsx           # sensitivity ⋆
│   │   │   ├── wallet/page.tsx            # cumulative SHAP ⋆
│   │   │   ├── checkin/page.tsx           # 15-sec daily form
│   │   │   └── layout.tsx                 # nav + auth gate
│   │   ├── api/
│   │   │   └── whoop/callback/route.ts    # OAuth callback proxy
│   │   ├── demo/page.tsx                  # synthetic-data demo
│   │   ├── layout.tsx                     # root layout
│   │   ├── manifest.json                  # PWA manifest
│   │   └── globals.css
│   ├── components/
│   │   ├── ReceiptCard.tsx
│   │   ├── LedgerRow.tsx
│   │   ├── BalanceHeader.tsx
│   │   ├── ForecastChart.tsx
│   │   ├── ConfidenceLabel.tsx            # "early estimate" pill
│   │   └── ui/                            # shadcn components
│   ├── lib/
│   │   ├── api.ts                         # fetch wrappers
│   │   ├── auth.ts                        # supabase auth
│   │   └── format.ts                      # number → "-9 pts"
│   ├── public/
│   │   ├── icon-192.png
│   │   └── icon-512.png
│   ├── next.config.js
│   ├── package.json
│   └── tsconfig.json
├── backend/                               # FastAPI
│   ├── api/
│   │   ├── main.py                        # FastAPI app entrypoint
│   │   ├── auth.py                        # JWT verification
│   │   ├── whoop.py                       # OAuth + webhook + sync
│   │   ├── checkins.py                    # daily check-in CRUD
│   │   ├── predictions.py                 # today's receipt
│   │   ├── simulate.py                    # what-if forward
│   │   ├── goals.py                       # inverse planner ⋆
│   │   ├── profile.py                     # sensitivity profile ⋆
│   │   └── wallet.py                      # cumulative SHAP ⋆
│   ├── ml/
│   │   ├── features.py                    # build_feature_matrix (PURE)
│   │   ├── train.py                       # Ridge + TimeSeriesSplit
│   │   ├── explain.py                     # shap.LinearExplainer wrapper
│   │   ├── solve.py                       # scipy.optimize ⋆
│   │   └── artifacts/                     # pickled pipelines (gitignored)
│   ├── workers/
```

```
│   │   ├── safety_pull.py                # 4 AM safety net cron
│   │   ├── nightly_train.py              # nightly cron entrypoint
│   │   └── push_forecast.py              # send PWA notif
│   ├── db/
│   │   ├── schema.sql                    # all 11 tables
│   │   ├── seeds.sql                     # demo synthetic data
│   │   └── client.py                     # asyncpg pool
│   ├── synth/
│   │   └── generator.py                  # realistic synthetic WHOOP data
│   ├── tests/
│   │   ├── test_features.py
│   │   ├── test_shap_integrity.py        # SHAP sums = prediction
│   │   ├── test_solve.py
│   │   └── test_synth_covariance.py
│   ├── requirements.txt
│   ├── pyproject.toml
│   └── .env.example
├── docs/                                 # source materials (already exists)
├── PLAN.md                               # daily checklist (already exists)
├── BUILD_GUIDE.md                        # this file
├── Recovery_Debt_PRD.md                  # the spec
├── .gitignore
└── README.md                             # public-facing
```

★ = Tier-1 differentiation feature from PRD §16

---

4. Environment Variables

`frontend/.env.local`

```
NEXT_PUBLIC_SUPABASE_URL=https://xxxxxxxxxxx.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY=eyJhbGciOi...
NEXT_PUBLIC_API_URL=http://localhost:8000        # Railway URL in prod
NEXT_PUBLIC_VAPID_PUBLIC_KEY=...                  # for push notifs (week 3)
```

`backend/.env`

```
DATABASE_URL=postgresql://postgres:xxx@db.xxxxxxxxxxx.supabase.co:5432/postgres
SUPABASE_JWT_SECRET=...                           # to verify frontend JWTs
WHOOP_CLIENT_ID=...
WHOOP_CLIENT_SECRET=...
WHOOP_REDIRECT_URI=https://yourapp.vercel.app/api/whoop/callback
WHOOP_WEBHOOK_SECRET=...
ANTHROPIC_API_KEY=sk-ant-...                      # week 3 stretch
VAPID_PRIVATE_KEY=...                             # week 3 push notifs
USER_TIMEZONE=America/New_York                    # for the 4 AM cron
```

Generate VAPID keys (for PWA push):

```
npx web-push generate-vapid-keys
```

---

5. Day-by-Day Build (with code)

Each day has a **goal**, **files to touch**, **commands**, and **a verification step**. Code blocks are skeletons — fill in details as you go.

Day 1 — Project skeletons

Goal: all three services boot locally.

```
mkdir recovery-debt && cd recovery-debt
git init

# Frontend
npx create-next-app@latest frontend --ts --tailwind --app --src-dir=false --import-alias="@/*"
cd frontend && npx shadcn-ui@latest init && cd ..

# Backend
mkdir backend && cd backend
python -m venv .venv && source .venv/bin/activate
pip install fastapi uvicorn[standard] httpx asyncpg pandas numpy scikit-learn shap scipy authlib
pytest ruff python-dotenv
pip freeze > requirements.txt
cd ..

# .gitignore
cat > .gitignore <<EOF
node_modules/
.next/
.env
.env.local
__pycache__/
.venv/
backend/ml/artifacts/
*.pyc
.DS_Store
EOF

git add . && git commit -m "init"
gh repo create recovery-debt --public --source=. --push
```

Backend skeleton — `backend/api/main.py`:

```python
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware

app = FastAPI(title="Recovery Debt API")

app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:3000", "https://*.vercel.app"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

@app.get("/health")
async def health():
    return {"ok": True}
```

Run both:

```bash
# terminal 1
cd backend && uvicorn api.main:app --reload --port 8000

# terminal 2
cd frontend && npm run dev
```

Verify: `curl localhost:8000/health` returns `{"ok":true}` and `localhost:3000` shows the Next.js welcome page.

Day 2 — Database schema + WHOOP OAuth

Schema — `backend/db/schema.sql` (all tables from PRD §8 + new `goals`):

```sql
-- Run this in Supabase SQL editor

CREATE EXTENSION IF NOT EXISTS "uuid-ossp";

CREATE TABLE users (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  whoop_user_id BIGINT UNIQUE,
  email TEXT,
  timezone TEXT DEFAULT 'America/New_York',
  created_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE whoop_tokens (
  user_id UUID PRIMARY KEY REFERENCES users(id) ON DELETE CASCADE,
  access_token TEXT NOT NULL,
  refresh_token TEXT NOT NULL,
  expires_at TIMESTAMPTZ NOT NULL
);

CREATE TABLE recoveries (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  user_id UUID REFERENCES users(id) ON DELETE CASCADE,
  day DATE NOT NULL,
  recovery_score INT,
  hrv_rmssd_ms FLOAT,
  rhr_bpm INT,
  spo2_pct FLOAT,
  skin_temp_c FLOAT,
  score_state TEXT,
  created_at TIMESTAMPTZ DEFAULT NOW(),
  UNIQUE (user_id, day)
);

-- Repeat the same UNIQUE pattern for cycles, sleeps, workouts, checkins.
-- (Full DDL in PRD §8 — copy directly from there.)

CREATE TABLE predictions (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  user_id UUID REFERENCES users(id) ON DELETE CASCADE,
  target_day DATE NOT NULL,
  predicted_recovery FLOAT NOT NULL,
  prediction_lower FLOAT,
  prediction_upper FLOAT,
  model_version TEXT NOT NULL,
  created_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE shap_values (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  prediction_id UUID REFERENCES predictions(id) ON DELETE CASCADE,
  feature_name TEXT NOT NULL,
  contribution FLOAT NOT NULL
);

CREATE TABLE models (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
```

```sql
  user_id UUID REFERENCES users(id) ON DELETE CASCADE,
  version TEXT NOT NULL,
  trained_at TIMESTAMPTZ DEFAULT NOW(),
  n_training_days INT,
  metrics JSONB,
  artifact_path TEXT
);

-- New table for the Inverse Planner (PRD §16 Tier-1 Feature A):
CREATE TABLE goals (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  user_id UUID REFERENCES users(id) ON DELETE CASCADE,
  target_day DATE NOT NULL,
  target_recovery FLOAT NOT NULL,
  solved_plan JSONB,
  infeasibility_reason TEXT,
  created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Indexes for hot paths
CREATE INDEX idx_recoveries_user_day ON recoveries(user_id, day DESC);
CREATE INDEX idx_predictions_user_day ON predictions(user_id, target_day DESC);
CREATE INDEX idx_shap_prediction ON shap_values(prediction_id);
```

WHOOP OAuth — `backend/api/whoop.py`:

```python
from fastapi import APIRouter, Depends, HTTPException, Request
from fastapi.responses import RedirectResponse
import httpx, os, secrets
from urllib.parse import urlencode

router = APIRouter(prefix="/api/whoop")

WHOOP_AUTH_URL = "https://api.prod.whoop.com/oauth/oauth2/auth"
WHOOP_TOKEN_URL = "https://api.prod.whoop.com/oauth/oauth2/token"

@router.get("/connect")
async def connect():
    state = secrets.token_urlsafe(16)
    params = {
        "client_id": os.environ["WHOOP_CLIENT_ID"],
        "redirect_uri": os.environ["WHOOP_REDIRECT_URI"],
        "response_type": "code",
        "scope": "read:recovery read:cycles read:sleep read:workout read:profile offline",
        "state": state,
    }
    return RedirectResponse(f"{WHOOP_AUTH_URL}?{urlencode(params)}")

@router.get("/callback")
async def callback(code: str, state: str):
    async with httpx.AsyncClient() as client:
        resp = await client.post(WHOOP_TOKEN_URL, data={
            "grant_type": "authorization_code",
            "code": code,
            "client_id": os.environ["WHOOP_CLIENT_ID"],
            "client_secret": os.environ["WHOOP_CLIENT_SECRET"],
            "redirect_uri": os.environ["WHOOP_REDIRECT_URI"],
        })
    if resp.status_code != 200:
        raise HTTPException(401, "WHOOP token exchange failed")
    tokens = resp.json()
    # TODO: save to whoop_tokens table — implement on Day 3
    return {"connected": True}
```

Verify: click "Connect WHOOP" on the frontend, get redirected to WHOOP, approve, land back on your site with `connected: true`.

Day 3 — Backfill 6 months

`backend/api/whoop.py` (continued):

```python
import asyncio
from datetime import datetime, timedelta

async def backfill(user_id: str, access_token: str):
    six_months_ago = (datetime.utcnow() - timedelta(days=180)).isoformat() + "Z"
    headers = {"Authorization": f"Bearer {access_token}"}

    async with httpx.AsyncClient(headers=headers) as client:
        for endpoint, table in [
            ("/v1/recovery", "recoveries"),
            ("/v1/cycle", "cycles"),
            ("/v1/activity/sleep", "sleeps"),
            ("/v1/activity/workout", "workouts"),
        ]:
            next_token = None
            while True:
                params = {"start": six_months_ago, "limit": 25}
                if next_token:
                    params["nextToken"] = next_token
                r = await client.get(f"https://api.prod.whoop.com{endpoint}", params=params)
                data = r.json()
                # TODO: insert each record into the corresponding table
                # using ON CONFLICT (user_id, day) DO UPDATE
                next_token = data.get("next_token")
                if not next_token:
                    break
                await asyncio.sleep(0.2)  # rate-limit politeness
```

Verify: open Supabase Studio, see ~180 rows in each of `recoveries`, `cycles`, `sleeps`. (Workouts will be fewer.)

Day 4 — Webhooks + safety-net cron

`backend/api/whoop.py`:

```python
import hmac, hashlib

@router.post("/webhook")
async def webhook(request: Request):
    body = await request.body()
    signature = request.headers.get("X-WHOOP-Signature", "")
    expected = hmac.new(
        os.environ["WHOOP_WEBHOOK_SECRET"].encode(),
        body,
        hashlib.sha256,
    ).hexdigest()
    if not hmac.compare_digest(signature, expected):
        raise HTTPException(401, "bad signature")

    payload = await request.json()
    event = payload["type"]      # "recovery.updated", "sleep.updated", etc.
    user_id = payload["user_id"]
    # TODO: queue a re-pull of just this user's last 3 days
    return {"ok": True}
```

Subscribe via the WHOOP developer portal. Manually fire a test webhook; confirm it lands.

Safety-net cron — `backend/workers/safety_pull.py`:

```python
"""Runs daily at 4 AM. Re-fetches last 3 days for every user."""
import asyncio
from db.client import db
from api.whoop import backfill_recent

async def main():
    users = await db.fetch("SELECT id, access_token FROM users JOIN whoop_tokens ON ...")
    for u in users:
        await backfill_recent(u["id"], u["access_token"], days=3)

if __name__ == "__main__":
    asyncio.run(main())
```

Hook it up on Railway: **Settings → Cron →** `0 4 * * *` **→** `python -m workers.safety_pull`.

Day 5 — First chart + early Vercel/Railway deploy

`frontend/app/(dashboard)/page.tsx`:

```
import { LineChart, Line, XAxis, YAxis, Tooltip, ResponsiveContainer } from "recharts";
import { fetchRecoveries } from "@/lib/api";

export default async function Dashboard() {
  const data = await fetchRecoveries();   // last 180 days
  return (
    <main className="p-6">
      <h1 className="text-2xl font-bold mb-4">Recovery — last 6 months</h1>
      <div className="h-72">
        <ResponsiveContainer>
          <LineChart data={data}>
            <XAxis dataKey="day" />
            <YAxis domain={[0, 100]} />
            <Tooltip />
            <Line type="monotone" dataKey="recovery_score" stroke="#1d4ed8" />
          </LineChart>
        </ResponsiveContainer>
      </div>
    </main>
  );
}
```

Deploy now so deploy bugs surface early:

```
# Frontend
cd frontend && vercel
# follow prompts; set NEXT_PUBLIC_API_URL to your future Railway URL

# Backend
cd backend
# push to GitHub, then in Railway: New Project → Deploy from GitHub
# add env vars, set start command: uvicorn api.main:app --host 0.0.0.0 --port $PORT
```

Friday-week-1 verify: open the live Vercel URL, log in, see your recovery line chart over 6 months. **Screenshot it for the README.**

Day 6 — Daily check-in form

`frontend/app/(dashboard)/checkin/page.tsx`:

```
"use client";
import { useState } from "react";
import { Slider } from "@/components/ui/slider";
import { Switch } from "@/components/ui/switch";
import { Button } from "@/components/ui/button";

export default function Checkin() {
  const [alcohol, setAlcohol] = useState(0);
  const [stress, setStress] = useState(5);
  const [lateMeal, setLateMeal] = useState(false);
  const [ill, setIll] = useState(false);
  const [traveling, setTraveling] = useState(false);

  async function submit() {
    await fetch("/api/checkin", {
      method: "POST",
      body: JSON.stringify({ alcohol_drinks: alcohol, stress_1to10: stress,
                             late_meal: lateMeal, ill, traveling }),
    });
  }

  return (
    <main className="p-6 max-w-md mx-auto space-y-6">
      <h1 className="text-2xl font-bold">Yesterday in 15 seconds</h1>
      <Field label={`Alcohol drinks: ${alcohol}`}>
        <Slider min={0} max={6} step={1} value={[alcohol]} onValueChange={(v) => setAlcohol(v[0])}
/>
      </Field>
      <Field label={`Stress: ${stress}/10`}>
        <Slider min={1} max={10} value={[stress]} onValueChange={(v) => setStress(v[0])} />
      </Field>
      <Field label="Ate within 3h of bed?"><Switch checked={lateMeal} onCheckedChange={setLateMeal}
/></Field>
      <Field label="Sick?"><Switch checked={ill} onCheckedChange={setIll} /></Field>
      <Field label="Traveling?"><Switch checked={traveling} onCheckedChange={setTraveling} />
</Field>
      <Button onClick={submit} className="w-full">Save</Button>
    </main>
  );
}

function Field({ label, children }: any) {
  return <div><div className="mb-1 text-sm font-medium">{label}</div>{children}</div>;
}
```

Backend handler — `backend/api/checkins.py`:

```python
from datetime import date

@router.post("/api/checkin")
async def upsert_checkin(payload: CheckinIn, user_id: str = Depends(current_user)):
    await db.execute("""
        INSERT INTO checkins (user_id, day, alcohol_drinks, caffeine_mg, stress_1to10,
                              late_meal, ill, traveling)
        VALUES ($1, $2, $3, $4, $5, $6, $7, $8)
        ON CONFLICT (user_id, day) DO UPDATE SET
            alcohol_drinks = EXCLUDED.alcohol_drinks,
            stress_1to10   = EXCLUDED.stress_1to10,
            late_meal      = EXCLUDED.late_meal,
            ill            = EXCLUDED.ill,
            traveling      = EXCLUDED.traveling
    """, user_id, date.today(), payload.alcohol_drinks, payload.caffeine_mg,
       payload.stress_1to10, payload.late_meal, payload.ill, payload.traveling)
```

Day 7 — Feature engineering (the pure function)

`backend/ml/features.py` — IMPORTANT: must be a pure function with no hidden state:

```python
import pandas as pd
import numpy as np
from typing import Optional

FEATURE_COLUMNS = [
    # Bucket A — last night's sleep
    "sleep_duration_h", "sleep_efficiency", "sleep_consistency",
    "rem_minutes", "deep_minutes", "disturbance_count",
    "respiratory_rate", "sleep_debt_3day",
    # Bucket B — yesterday's behavior
    "alcohol_drinks_lag1", "caffeine_mg_lag1", "stress_lag1", "late_meal_lag1",
    "meeting_minutes_lag1", "meetings_after_8pm_lag1", "travel_lag1", "ill_lag1",
    # Bucket C — yesterday's physiology
    "strain_lag1", "avg_hr_lag1", "recovery_lag1", "hrv_lag1", "rhr_lag1",
    # Bucket D — recent trends
    "strain_3day_avg", "strain_7day_avg", "hrv_7day_avg", "rhr_7day_avg",
    "recovery_3day_avg", "hrv_zscore_28d", "rhr_zscore_28d",
    # Bucket E — external (optional)
    "weather_temp_max_c", "weather_pressure_hpa",
]


def build_feature_matrix(
    recoveries: pd.DataFrame,
    sleeps: pd.DataFrame,
    cycles: pd.DataFrame,
    workouts: pd.DataFrame,
    checkins: pd.DataFrame,
    weather: Optional[pd.DataFrame] = None,
    *,
    target_days: Optional[pd.DatetimeIndex] = None,
    overrides: Optional[dict] = None,   # {date: {"sleep_duration_h": 7.5}}
) -> pd.DataFrame:
    """
    Pure function. No DB calls, no global state, no I/O.

    Returns one row per day with FEATURE_COLUMNS + 'recovery' (label) + 'day' (index).
    The `overrides` parameter is what makes counterfactual replay possible later.
    """
    # 1. Index everything by day
    df = pd.DataFrame(index=target_days or pd.date_range(...))

    # 2. Sleep features (current night → today's row)
    df["sleep_duration_h"] = sleeps["in_bed_ms"] / 3.6e6
    df["sleep_efficiency"] = sleeps["efficiency_pct"] / 100
    # ... etc

    # 3. Lagged features (yesterday → today)
    df["alcohol_drinks_lag1"] = checkins["alcohol_drinks"].shift(1)
    df["strain_lag1"]         = cycles["strain"].shift(1)
    # ... etc

    # 4. Rolling windows
    df["strain_3day_avg"] = cycles["strain"].rolling(3).mean()
    df["hrv_7day_avg"]    = recoveries["hrv_rmssd_ms"].rolling(7).mean()
```

```python
    # 5. Z-scores
    hrv_28 = recoveries["hrv_rmssd_ms"].rolling(28)
    df["hrv_zscore_28d"] = (recoveries["hrv_rmssd_ms"].shift(1) - hrv_28.mean()) / hrv_28.std()

    # 6. Missing-flags
    for col in FEATURE_COLUMNS:
        miss_pct = df[col].isna().mean()
        if miss_pct > 0.10:
            df[f"was_missing_{col}"] = df[col].isna().astype(int)

    # 7. Apply overrides (for counterfactual replay)
    if overrides:
        for d, vals in overrides.items():
            for k, v in vals.items():
                df.at[d, k] = v

    df["recovery"] = recoveries["recovery_score"]
    return df.dropna(subset=["recovery"])
```

Test it:

```python
# backend/tests/test_features.py
def test_build_feature_matrix_is_pure():
    """Same input → same output, every time."""
    a = build_feature_matrix(rec_df, sleep_df, cycle_df, work_df, ck_df)
    b = build_feature_matrix(rec_df, sleep_df, cycle_df, work_df, ck_df)
    pd.testing.assert_frame_equal(a, b)

def test_build_feature_matrix_handles_overrides():
    base = build_feature_matrix(rec_df, sleep_df, cycle_df, work_df, ck_df)
    cf = build_feature_matrix(rec_df, sleep_df, cycle_df, work_df, ck_df,
                              overrides={pd.Timestamp("2026-04-15"): {"sleep_duration_h": 5.0}})
    assert cf.at[pd.Timestamp("2026-04-15"), "sleep_duration_h"] == 5.0
```

Day 8 — Train Ridge

`backend/ml/train.py`:

```python
import math, pickle, datetime as dt
from pathlib import Path
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.linear_model import RidgeCV
from sklearn.model_selection import TimeSeriesSplit
from sklearn.metrics import r2_score, mean_squared_error, mean_absolute_error
from .features import build_feature_matrix, FEATURE_COLUMNS

MIN_TRAINING_DAYS = 21
ARTIFACT_DIR = Path(__file__).parent / "artifacts"
ARTIFACT_DIR.mkdir(exist_ok=True)


def nightly_train(user_id: str, dfs: dict) -> dict | None:
    df = build_feature_matrix(**dfs)

    if len(df) < MIN_TRAINING_DAYS:
        return {"skipped": True, "reason": "cold_start", "n_days": len(df)}

    val_size = max(7, int(0.2 * len(df)))
    train, val = df.iloc[:-val_size], df.iloc[-val_size:]

    feature_cols = [c for c in df.columns if c != "recovery"]
    X_train, y_train = train[feature_cols], train["recovery"]
    X_val,   y_val   = val[feature_cols],   val["recovery"]

    pipeline = Pipeline([
        ("impute", SimpleImputer(strategy="median")),
        ("scale",  StandardScaler()),
        ("ridge",  RidgeCV(
            alphas=[0.1, 1.0, 10.0, 100.0],
            cv=TimeSeriesSplit(n_splits=5),
        )),
    ])
    pipeline.fit(X_train, y_train)
    pred_val = pipeline.predict(X_val)

    metrics = {
        "r2":   float(r2_score(y_val, pred_val)),
        "rmse": math.sqrt(mean_squared_error(y_val, pred_val)),
        "mae":  float(mean_absolute_error(y_val, pred_val)),
        "n_train": len(X_train),
        "n_val":   len(X_val),
    }

    # Refit on full window for production
    pipeline.fit(df[feature_cols], df["recovery"])

    version = dt.date.today().isoformat()
    artifact = ARTIFACT_DIR / f"{user_id}_{version}.pkl"
    with open(artifact, "wb") as f:
        pickle.dump({"pipeline": pipeline, "feature_cols": feature_cols}, f)
```

```
    return {"version": version, "metrics": metrics, "artifact": str(artifact)}
```

Verify: $R^2 > 0.3$ is fine for a single-user model with limited data; RMSE < 12 is good.

Day 9 — SHAP receipts

`backend/ml/explain.py`:

```
import shap
import numpy as np

def build_explainer(pipeline, X_full):
    """LinearExplainer is exact (not approximate) on Ridge."""
    pre = pipeline[:-1]            # impute + scale
    ridge = pipeline.named_steps["ridge"]
    background = pre.transform(X_full)
    return shap.LinearExplainer(ridge, background)

def explain_one(pipeline, explainer, x_row, feature_cols):
    pre = pipeline[:-1]
    x_scaled = pre.transform(x_row.reshape(1, -1))
    shap_vals = explainer.shap_values(x_scaled)[0]
    base = float(explainer.expected_value)
    contribs = sorted(
        zip(feature_cols, shap_vals.tolist()),
        key=lambda t: abs(t[1]),
        reverse=True,
    )
    return {"base_value": base, "contributions": contribs[:5]}  # top 5
```

Integrity test — `backend/tests/test_shap_integrity.py`:

```
def test_shap_sums_to_prediction():
    """base + sum(all contributions) must equal prediction within 0.01."""
    pred = pipeline.predict(x_row.reshape(1, -1))[0]
    base = explainer.expected_value
    all_shap = explainer.shap_values(pre.transform(x_row.reshape(1, -1)))[0]
    assert abs(base + all_shap.sum() - pred) < 0.01, "SHAP integrity broken"
```

Day 10 — Nightly cron + receipt UI

`backend/workers/nightly_train.py`:

```python
"""Cron at 4 AM local. Train, predict tomorrow, store SHAP."""
import asyncio
from datetime import date, timedelta
from db.client import db
from ml.train import nightly_train
from ml.explain import build_explainer, explain_one
from ml.features import build_feature_matrix, FEATURE_COLUMNS

async def main():
    users = await db.fetch("SELECT id FROM users")
    for u in users:
        dfs = await load_user_dataframes(u["id"])
        result = nightly_train(u["id"], dfs)
        if result.get("skipped"):
            continue

        # Save model row
        model_id = await db.fetchval("""
            INSERT INTO models (user_id, version, n_training_days, metrics, artifact_path)
            VALUES ($1, $2, $3, $4, $5) RETURNING id
        """, u["id"], result["version"], result["metrics"]["n_train"],
            json.dumps(result["metrics"]), result["artifact"])

        # Predict tomorrow + SHAP
        tomorrow = date.today() + timedelta(days=1)
        pipeline = load_pipeline(result["artifact"])
        df = build_feature_matrix(**dfs, target_days=[tomorrow])
        explainer = build_explainer(pipeline, df[FEATURE_COLUMNS].values)

        x = df[FEATURE_COLUMNS].iloc[-1].values
        pred = float(pipeline.predict(x.reshape(1, -1))[0])
        explanation = explain_one(pipeline, explainer, x, FEATURE_COLUMNS)

        # Persist
        pred_id = await db.fetchval("""
            INSERT INTO predictions (user_id, target_day, predicted_recovery, model_version)
            VALUES ($1, $2, $3, $4) RETURNING id
        """, u["id"], tomorrow, pred, result["version"])

        for fname, contrib in explanation["contributions"]:
            await db.execute("""
                INSERT INTO shap_values (prediction_id, feature_name, contribution)
                VALUES ($1, $2, $3)
            """, pred_id, fname, contrib)

if __name__ == "__main__":
    asyncio.run(main())
```

Receipt card — `frontend/components/ReceiptCard.tsx`:

```
type Contribution = { feature: string; contribution: number };

export function ReceiptCard({ score, contributions, isEarlyEstimate }:
  { score: number; contributions: Contribution[]; isEarlyEstimate: boolean }) {
  return (
    <div className="border rounded-lg p-4 space-y-2 bg-white">
      <div className="flex items-baseline justify-between">
        <h2 className="font-bold text-lg">Today's receipt</h2>
        <span className="text-3xl font-bold">{score}</span>
      </div>
      {isEarlyEstimate && (
        <div className="text-xs text-amber-600">
          Early estimate — model still learning ({"<"} 60 days of data)
        </div>
      )}
      <ul className="space-y-1 text-sm">
        {contributions.map((c) => (
          <li key={c.feature} className="flex justify-between">
            <span>{humanize(c.feature)}</span>
            <span className={c.contribution >= 0 ? "text-green-700" : "text-red-700"}>
              {c.contribution >= 0 ? "+" : ""}{c.contribution.toFixed(1)} pts
            </span>
          </li>
        ))}
      </ul>
    </div>
  );
}

function humanize(f: string): string {
  return f.replace(/_/g, " ").replace(/lag1/, "yesterday")
          .replace(/3day_avg/, "3-day avg");
}
```

Friday-week-2 verify: open the app and read your receipt: *"Sleep yesterday: −10. Alcohol yesterday: −6. Late meeting: −2."* Real numbers from your real life.

Day 11 — The Ledger UI (banking style)

```
// frontend/app/(dashboard)/page.tsx
import { fetchLedger } from "@/lib/api";
import { LedgerRow } from "@/components/LedgerRow";
import { BalanceHeader } from "@/components/BalanceHeader";

export default async function Ledger() {
  const { balance, forecast, days } = await fetchLedger();
  return (
    <main className="max-w-2xl mx-auto p-4">
      <BalanceHeader balance={balance} forecast={forecast} />
      <div className="mt-6 divide-y">
        {days.map((d) => <LedgerRow key={d.day} {...d} />)}
      </div>
    </main>
  );
}
```

```
// frontend/components/LedgerRow.tsx
export function LedgerRow({ day, score, delta, top_contributors }: any) {
  const sign = delta >= 0 ? "+" : "";
  const color = delta >= 0 ? "text-green-700" : "text-red-700";
  return (
    <div className="py-3 flex items-baseline justify-between">
      <div>
        <div className="font-medium">{day}</div>
        <div className="text-xs text-gray-500">
          {top_contributors.map((c: any) => `${c.feature} ${c.contribution.toFixed(0)}`).join(" ·
")}
        </div>
      </div>
      <div className="text-right">
        <div className="font-bold text-lg">{score}</div>
        <div className={`text-sm ${color}`}>{sign}{delta.toFixed(0)} pts</div>
      </div>
    </div>
  );
}
```

Backend — recovery debt math (PRD §10):

```python
# backend/api/predictions.py
@router.get("/api/ledger")
async def ledger(user_id: str = Depends(current_user)):
    rows = await db.fetch("""
        SELECT day, recovery_score FROM recoveries
        WHERE user_id = $1 ORDER BY day DESC LIMIT 28
    """, user_id)
    scores = [r["recovery_score"] for r in rows]
    baseline = sorted(scores)[len(scores)//2]      # median
    last7 = scores[:7]
    debt_balance = -sum(s - baseline for s in last7)
    # forecast comes from the predictions table
    return {"balance": debt_balance, "baseline": baseline, "days": rows}
```

Day 12 — What-If Simulator

```tsx
// frontend/app/(dashboard)/simulator/page.tsx
"use client";
import { useState, useEffect } from "react";
import { Slider } from "@/components/ui/slider";

export default function Simulator() {
  const [sleep, setSleep] = useState(7);
  const [strain, setStrain] = useState(12);
  const [forecast, setForecast] = useState<number[]>([]);

  useEffect(() => {
    const t = setTimeout(async () => {
      const r = await fetch("/api/simulate", {
        method: "POST",
        body: JSON.stringify({ tomorrow_sleep: sleep, tomorrow_strain: strain }),
      });
      const data = await r.json();
      setForecast(data.forecast);
    }, 250);  // debounce
    return () => clearTimeout(t);
  }, [sleep, strain]);

  return (
    <main className="p-6 max-w-md mx-auto space-y-6">
      <h1 className="text-2xl font-bold">What if…</h1>
      <Field label={`Tomorrow's sleep: ${sleep}h`}>
        <Slider min={4} max={10} step={0.5} value={[sleep]} onValueChange={(v) => setSleep(v[0])} />
      </Field>
      <Field label={`Tomorrow's strain: ${strain}`}>
        <Slider min={0} max={21} step={0.5} value={[strain]} onValueChange={(v) => setStrain(v[0])}
/>
      </Field>
      <ForecastBars forecast={forecast} />
    </main>
  );
}
```

Backend just calls `pipeline.predict()` with the user's `x_baseline` modified by the sliders.

Day 13 — Inverse Planner ★

This is the killer. See Section 6.A for the full implementation.

Day 14 — Sensitivity Profile + SHAP Wallet ★

See Sections 6.B and 6.C.

Day 15 — PWA + Demo Mode + Deploy + Loom

PWA — `frontend/next.config.js`:

```
const withPWA = require("next-pwa")({
  dest: "public",
  register: true,
  skipWaiting: true,
});
module.exports = withPWA({ reactStrictMode: true });
```

`frontend/public/manifest.json`:

```
{
  "name": "Recovery Debt",
  "short_name": "Recovery",
  "icons": [
    { "src": "/icon-192.png", "sizes": "192x192", "type": "image/png" },
    { "src": "/icon-512.png", "sizes": "512x512", "type": "image/png" }
  ],
  "start_url": "/",
  "display": "standalone",
  "background_color": "#ffffff",
  "theme_color": "#1d4ed8"
}
```

Demo mode — `backend/synth/generator.py`:

```python
"""Generate 180 days of realistic synthetic WHOOP data with proper covariance."""
import numpy as np
import pandas as pd

def generate_synthetic_user(seed=42, n_days=180):
    rng = np.random.default_rng(seed)
    days = pd.date_range(end=pd.Timestamp.today(), periods=n_days)

    sleep_h = rng.normal(7.2, 1.0, n_days).clip(4, 10)
    strain  = rng.normal(11, 4, n_days).clip(0, 21)
    alcohol = rng.poisson(0.6, n_days).clip(0, 6)
    stress  = rng.integers(2, 9, n_days)

    # Recovery is correlated with all of the above (the whole point)
    recovery = (
        50
        + 4 * (sleep_h - 7)
        - 2 * (strain - 11)
        - 3 * alcohol
        - 1.2 * (stress - 5)
        + rng.normal(0, 6, n_days)
    ).clip(0, 100).round().astype(int)

    return pd.DataFrame({
        "day": days, "sleep_h": sleep_h, "strain": strain,
        "alcohol": alcohol, "stress": stress, "recovery": recovery,
    })
```

Verify the synthetic covariance matters:

```python
def test_synth_has_realistic_covariance():
    df = generate_synthetic_user()
    assert df[["sleep_h", "recovery"]].corr().iloc[0,1] > 0.3
    assert df[["alcohol", "recovery"]].corr().iloc[0,1] < -0.2
```

Loom checklist (60–90 sec): 1. Open live URL → ledger 2. Click today → SHAP receipt 3. Open simulator → drag sleep slider → forecast cascades 4. Open `/plan` → type "75 by Saturday" → plan appears 5. Open `/profile` → show coefficient bars 6. Open `/wallet` → show cumulative attribution 7. End on the surprising real insight from your data

---

6. The 3 Differentiation Features (in detail)

6.A Inverse Planner

The pitch (recruiter line): *"I built a constrained optimizer on top of my own ML model. Given a recovery target, it solves for the required sleep, strain, and behaviors via SLSQP — and tells you which constraint binds when the target is infeasible."*

`backend/ml/solve.py`:

```python
import numpy as np
from scipy.optimize import minimize, NonlinearConstraint

# Physiological feasibility (these are HARD bounds)
BOUNDS = {
    "sleep_duration_h":   (5.0, 10.0),
    "alcohol_drinks_lag1": (0,    6),
    "stress_lag1":        (1,    10),
    "strain_lag1":        (0,    21),
    "late_meal_lag1":     (0,    1),
    # ... others as appropriate
}

CONTROLLABLE = list(BOUNDS.keys())


def solve_for_target(
    pipeline,                                       # trained Ridge pipeline
    feature_cols: list[str],
    x_baseline: np.ndarray,                         # user's "typical day" feature row
    target_recovery: float,
) -> dict:
    """Solve: min ||x − x_baseline||² s.t. β·x ≥ y* and physio bounds."""

    # Index of each controllable feature in the full feature row
    idx = {f: feature_cols.index(f) for f in CONTROLLABLE if f in feature_cols}

    def predict(x_full):
        return pipeline.predict(x_full.reshape(1, -1))[0]

    def objective(x_ctrl):
        x_full = x_baseline.copy()
        for f, j in idx.items():
            x_full[j] = x_ctrl[list(idx).index(f)]
        deviation = (x_full - x_baseline) ** 2
        return float(deviation.sum())

    def recovery_constraint(x_ctrl):
        x_full = x_baseline.copy()
        for f, j in idx.items():
            x_full[j] = x_ctrl[list(idx).index(f)]
        return predict(x_full) - target_recovery

    x0 = np.array([x_baseline[idx[f]] for f in CONTROLLABLE if f in idx])
    bounds = [BOUNDS[f] for f in CONTROLLABLE if f in idx]

    result = minimize(
        objective,
        x0,
        method="SLSQP",
        bounds=bounds,
        constraints=[{"type": "ineq", "fun": recovery_constraint}],
        options={"maxiter": 50},
    )

    if not result.success or recovery_constraint(result.x) < -0.5:
```

```
        # Infeasible — find closest achievable
        max_pred = _maximize_recovery(pipeline, feature_cols, x_baseline, idx, bounds)
        return {
            "feasible": False,
            "closest_achievable": max_pred,
            "binding_constraint": _find_binding(result.x, bounds, list(idx)),
        }

    plan = {f: float(result.x[i]) for i, f in enumerate(idx.keys())}
    return {"feasible": True, "plan": plan, "predicted_recovery": float(predict(_pack(result.x,
x_baseline, idx)))}
```

API endpoint — `backend/api/goals.py`:

```
@router.post("/api/goals/solve")
async def solve_goal(g: GoalIn, user_id: str = Depends(current_user)):
    pipeline, feature_cols, x_baseline = await load_latest_artifacts(user_id)
    result = solve_for_target(pipeline, feature_cols, x_baseline, g.target_recovery)

    goal_id = await db.fetchval("""
        INSERT INTO goals (user_id, target_day, target_recovery,
                           solved_plan, infeasibility_reason)
        VALUES ($1, $2, $3, $4, $5) RETURNING id
    """, user_id, g.target_day, g.target_recovery,
        json.dumps(result.get("plan")) if result["feasible"] else None,
        result.get("binding_constraint"))

    return {"goal_id": goal_id, **result}
```

6.B Sensitivity Profile

The pitch: *"After 30 nightly retraining runs, I show the median + IQR of each Ridge coefficient. Stable coefficients = the model knows you. High-variance coefficients = the model is still learning. This entire page is impossible without a per-user model."*

`backend/api/profile.py`:

```python
@router.get("/api/profile/sensitivity")
async def sensitivity(user_id: str = Depends(current_user)):
    rows = await db.fetch("""
        SELECT artifact_path FROM models
        WHERE user_id = $1 ORDER BY trained_at DESC LIMIT 30
    """, user_id)

    coef_history = []  # list[dict[feature → standardized coef]]
    for r in rows:
        with open(r["artifact_path"], "rb") as f:
            blob = pickle.load(f)
        ridge = blob["pipeline"].named_steps["ridge"]
        feature_cols = blob["feature_cols"]
        coef_history.append(dict(zip(feature_cols, ridge.coef_.tolist())))

    df = pd.DataFrame(coef_history)
    summary = pd.DataFrame({
        "median": df.median(),
        "q25":    df.quantile(0.25),
        "q75":    df.quantile(0.75),
        "stable": (df.quantile(0.75) - df.quantile(0.25)) < df.std() * 0.5,
    })
    return summary.reset_index().rename(columns={"index": "feature"}).to_dict("records")
```

Frontend renders as a horizontal bar chart with whisker error bars.

6.C Cumulative SHAP Wallet

The pitch: *"Spotify Wrapped, but for your body. Cumulative attribution by feature category over the year. Coach has no concept of longitudinal attribution."*

Important caveat: SHAP values from different model versions are not directly comparable. Solution: re-explain all historical days through the *current* model nightly.

Schema addition:

```sql
CREATE TABLE feature_categories (
  feature_name TEXT PRIMARY KEY,
  category TEXT NOT NULL CHECK (category IN ('sleep', 'behavior', 'strain', 'external', 'trend'))
);

INSERT INTO feature_categories VALUES
  ('sleep_duration_h', 'sleep'),
  ('alcohol_drinks_lag1', 'behavior'),
  ('strain_lag1', 'strain'),
  ('weather_temp_max_c', 'external'),
  ('hrv_7day_avg', 'trend');
-- ... etc
```

Endpoint — `backend/api/wallet.py`:

```python
@router.get("/api/wallet")
async def wallet(user_id: str, days: int = 90):
    rows = await db.fetch("""
        SELECT p.target_day, fc.category, SUM(sv.contribution) AS contribution
        FROM predictions p
        JOIN shap_values sv ON sv.prediction_id = p.id
        JOIN feature_categories fc ON fc.feature_name = sv.feature_name
        WHERE p.user_id = $1
          AND p.target_day >= CURRENT_DATE - $2 * INTERVAL '1 day'
        GROUP BY p.target_day, fc.category
        ORDER BY p.target_day
    """, user_id, days)
    return [dict(r) for r in rows]
```

Frontend — Recharts area chart:

```xml
<ResponsiveContainer width="100%" height={400}>
  <AreaChart data={walletData}>
    <XAxis dataKey="target_day" />
    <YAxis />
    <Tooltip />
    <Area type="monotone" dataKey="sleep"    stackId="1" stroke="#10b981" fill="#10b981" />
    <Area type="monotone" dataKey="behavior" stackId="1" stroke="#ef4444" fill="#ef4444" />
    <Area type="monotone" dataKey="strain"   stackId="1" stroke="#f59e0b" fill="#f59e0b" />
    <Area type="monotone" dataKey="external" stackId="1" stroke="#6366f1" fill="#6366f1" />
    <Area type="monotone" dataKey="trend"    stackId="1" stroke="#8b5cf6" fill="#8b5cf6" />
  </AreaChart>
</ResponsiveContainer>
```

---

7. Deployment

Frontend — Vercel

```bash
cd frontend
vercel link              # connect to GitHub repo
vercel env add NEXT_PUBLIC_API_URL production
vercel env add NEXT_PUBLIC_SUPABASE_URL production
# ... etc
git push                 # auto-deploys
```

Backend — Railway

1. Railway → **New Project → Deploy from GitHub repo** → select `recovery-debt`, set root to `/backend`.
2. Add all `backend/.env` variables to Railway's variables panel.
3. **Settings → Start Command:** `uvicorn api.main:app --host 0.0.0.0 --port $PORT`
4. **Settings → Cron Jobs:** - `0 4 * * *` — `python -m workers.safety_pull` - `15 4 * * *` — `python -m workers.nightly_train` - `0 21 * * *` — `python -m workers.push_forecast`

Supabase

1. Run `db/schema.sql` in the SQL editor.
2. **Authentication → Providers → enable Email magic link** (or Google).
3. **Database → Replication → enable** if you want realtime subscriptions later.
4. **Row-Level Security:** add `auth.uid() = user_id` policies on all per-user tables.

---

8. Testing Strategy

Layer	Test	Why it matters
Pure features	`test_features.py::test_pure` — same input → same output	Counterfactual replay depends on this
SHAP integrity	`test_shap_integrity.py` — base + Σ contribs ≈ prediction	Catches a wrong-reference-data bug that silently breaks every receipt
Time-series split	`test_train.py::test_no_future_leakage` — assert val_idx > train_idx max	Random splits make models look great in val and useless in prod
Inverse planner	`test_solve.py::test_feasibility` — known-feasible target returns plan within bounds	Catches solver misconfiguration
Synthetic covariance	`test_synth_covariance.py` — sleep ↑ → recovery ↑, alcohol ↑ → recovery ↓	Demo mode looks insane otherwise
WHOOP webhook	manual: send a test webhook from WHOOP dev portal, assert it lands in DB	Webhook silent failures cost a day of data
Cold-start UI	E2E: new user with <60 days, every receipt shows "early estimate" pill	Honesty principle from PRD §13
Token refresh	unit: expired access_token → refresh → succeeds	WHOOP tokens expire in 1 hour

Run before every commit:

```
cd backend && pytest -x
cd frontend && npm run lint && npm run build
```

---

9. Common Errors & Fixes

Error	Cause	Fix
`ridge.coef_` is all zeros	Forgot `StandardScaler` step	Add it to the pipeline (PRD §12)
SHAP integrity test fails by ~0.5 pts	Built `LinearExplainer` on different reference data than `pipeline[:-1].transform(X)`	Use exact training-time transform
Webhook returns 401	Wrong HMAC computation	Use `hmac.compare_digest`, not `==`
`403` from WHOOP API	Access token expired	Refresh on the fly using `refresh_token`
Train-test split looks weird	Used random `KFold` instead of `TimeSeriesSplit`	NEVER random on time series — leaks the future
Inverse planner returns 10h sleep + max strain	Box constraints too loose	Tighten bounds; consider feature couplings as v2
Synthetic data → $R^2 \approx 0$	Generator has no real covariance	Make recovery explicitly depend on sleep/strain/alcohol with noise
PWA won't install on iOS	Missing `apple-touch-icon` and `theme-color` meta tags	Add to `<head>` in `app/layout.tsx`
Vercel build fails on `pickle` import	sklearn not in `requirements.txt` for any frontend code path	Frontend never loads pickle; ensure ML code is backend-only

---

10. References

- **WHOOP API docs** — https://developer.whoop.com/api/
- **WHOOP OAuth 2.0** — https://developer.whoop.com/docs/developing/authentication
- **WHOOP webhooks** — https://developer.whoop.com/docs/developing/webhooks
- **Supabase Postgres** — https://supabase.com/docs/guides/database
- **Supabase Auth** — https://supabase.com/docs/guides/auth
- **scikit-learn Pipeline** — https://scikit-learn.org/stable/modules/generated/sklearn.pipeline.Pipeline.html
- **scikit-learn TimeSeriesSplit** — https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.TimeSeriesSplit.html
- **shap.LinearExplainer** — https://shap.readthedocs.io/en/latest/example_notebooks/api_examples/explainers/LinearExplainer.html
- **scipy.optimize.minimize** — https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.minimize.html
- **Next.js App Router** — https://nextjs.org/docs/app
- **next-pwa** — https://github.com/shadowwalker/next-pwa
- **Recharts** — https://recharts.org/en-US/api
- **shadcn/ui** — https://ui.shadcn.com/
- **Web Push API (VAPID)** — https://developer.mozilla.org/en-US/docs/Web/API/Push_API
- **Anthropic Python SDK** — https://docs.anthropic.com/en/api/client-sdks

---

What This Build Demonstrates (your interview talking points)

1. **Real ML pipeline** — feature engineering, time-series CV, nightly retraining loop
2. **Model explainability** — SHAP with an integrity unit test
3. **Constrained optimization** — scipy `SLSQP` on top of a trained model (the inverse planner)
4. **Coefficient stability analysis** — IQR across model versions = production model-monitoring concept
5. **Longitudinal attribution aggregation** — SHAP as an aggregable signal, not just per-prediction
6. **Full-stack delivery** — OAuth 2.0, webhooks, signature verification, scheduled jobs, PWA, public deploy
7. **LLM as a tool, not the product** — strict-JSON parser, not a chatbot
8. **Honest framing of limits** — confidence intervals, "early estimate" labels, correlation-vs-causation discipline
9. **Testing discipline** — purity tests, SHAP integrity tests, time-series leakage tests
10. **Synthetic-data engineering** — preserving realistic covariance for demo mode

That's a lot of senior-grade signals in one project. Build it.